ARTIFICIAL INTELLIGENCE ASSIGNMENT

(Q1) With respect to the data set Provided, build the model using sequential Modelling Technique

1. Build the Open stock price prediction model using LSTM

2. Build the Close stock price prediction model using RNN

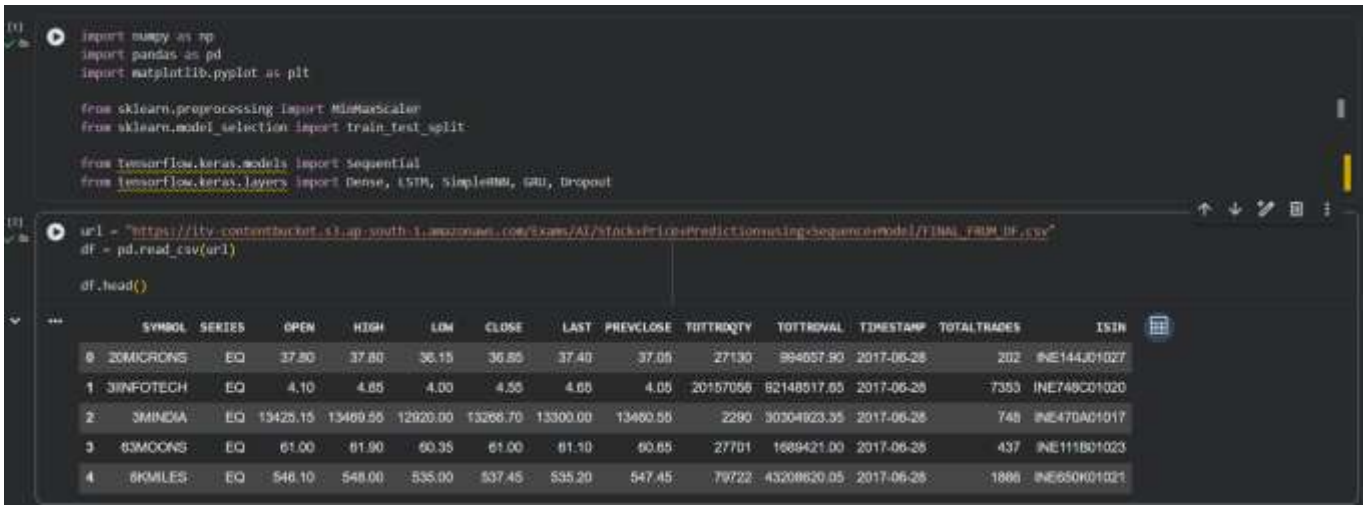
3. Take the reference of the Stock price prediction to build the models using GRU and analyse the results

Dataset Link : https://itv-contentbucket.s3.ap-south-1.amazonaws.com/Exams/AI/Stock+Price+Prediction+using+Sequence+Model/FINAL_FROM_DF.csv

(ANS) # CODE

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, SimpleRNN, GRU, Dropout


# LOAD DATA SET
url = "https://itv-contentbucket.s3.ap-south-1.amazonaws.com/Exams/AI/Stock+Price+Prediction+using+Sequence+Model/FINAL_FROM_DF.csv"
df = pd.read_csv(url)

df.head()
```

SCREEN SHOT



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, SimpleRNN, GRU, Dropout
```

```
url = "https://itv-contentbucket.s3.ap-south-1.amazonaws.com/Exams/AI/StockPricePredictionusingSequenceModel/FINAL_FROM_DF.csv"
df = pd.read_csv(url)

df.head()
```

	SYMBOL	SERIES	OPEN	HIGH	LOW	CLOSE	LAST	PREVCLOSE	TOTTRDQTY	TOTTRDVAL	TIMESTAMP	TOTALTRADES	ISIN
0	20MICRONS	EQ	37.80	37.80	36.15	36.85	37.40	37.05	27130	994657.90	2017-06-28	202	INE144J01027
1	3IINFOTECH	EQ	4.10	4.85	4.00	4.55	4.65	4.05	20157058	82148517.65	2017-06-28	7353	INE748C01020
2	3MINDIA	EQ	13425.15	13469.55	12920.00	13268.70	13300.00	13460.55	2290	30304923.35	2017-06-28	748	INE470A01017
3	63MOONS	EQ	61.00	61.90	60.35	61.00	61.10	60.65	27701	1689421.00	2017-06-28	437	INE111B01023
4	8KMILES	EQ	548.10	548.00	535.00	537.45	535.20	547.45	79722	43208620.05	2017-06-28	1886	INE650K01021

DATA PREPROCESSING

#SELECT FEATURES

CODE

df = df[['Open', 'Close']]

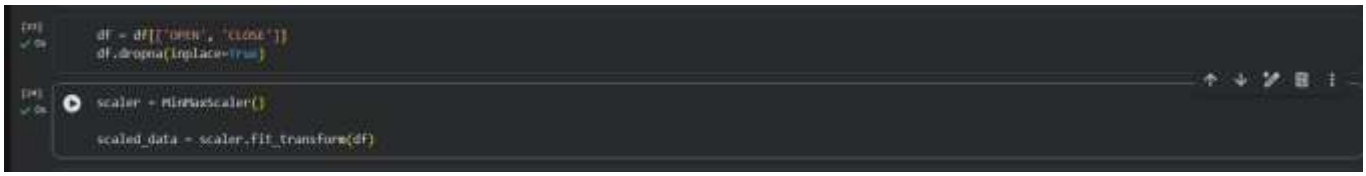
df.dropna(inplace=True)

NORMALIZE DATA

scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(df)

SCREENSHOT



```
df = df[['Open', 'Close']]
df.dropna(inplace=True)
```

```
scaler = MinMaxScaler()

scaled_data = scaler.fit_transform(df)
```

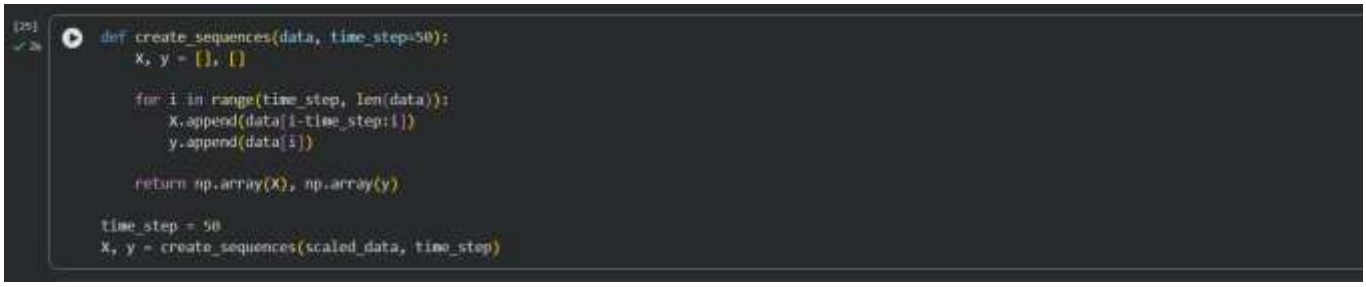
Create Sequences (Time Series Data)

code

def create_sequences(data, time_step=50):

```
    X, y = [], []

    for i in range(time_step, len(data)):
        X.append(data[i-time_step:i])
        y.append(data[i])

    return np.array(X), np.array(y)

time_step = 50
X, y = create_sequences(scaled_data, time_step)
```

screenshot



Train-Test Split

#code

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, shuffle=False)
```

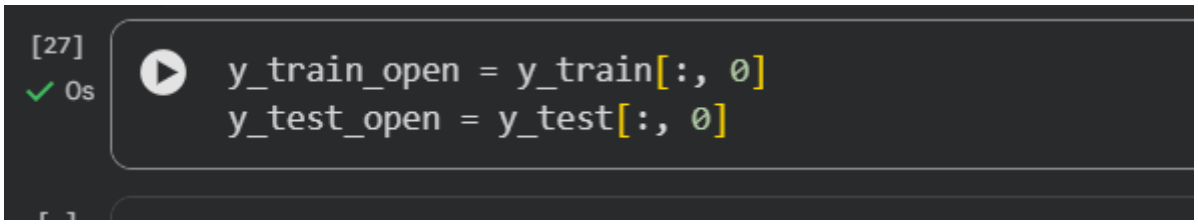
screenshot

#LSTM Model (Open Price Prediction)

#code

```
y_train_open = y_train[:, 0]
y_test_open = y_test[:, 0]
```

screenshot



#Build LSTM Model

code

model_lstm = Sequential()

model_lstm.add(LSTM(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])))

model_lstm.add(Dropout(0.2))

model_lstm.add(LSTM(64))

model_lstm.add(Dropout(0.2))

model_lstm.add(Dense(1))

model_lstm.compile(optimizer='adam', loss='mse')

screenshot

#Train

code

model_lstm.fit(X_train, y_train_open, epochs=20, batch_size=32)

screenshot

RNN Model (Close Price Prediction)

#code

y_train_close = y_train[:, 1]

y_test_close = y_test[:, 1]

#screenshot

model

#code

model_rnn = Sequential([

SimpleRNN(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),

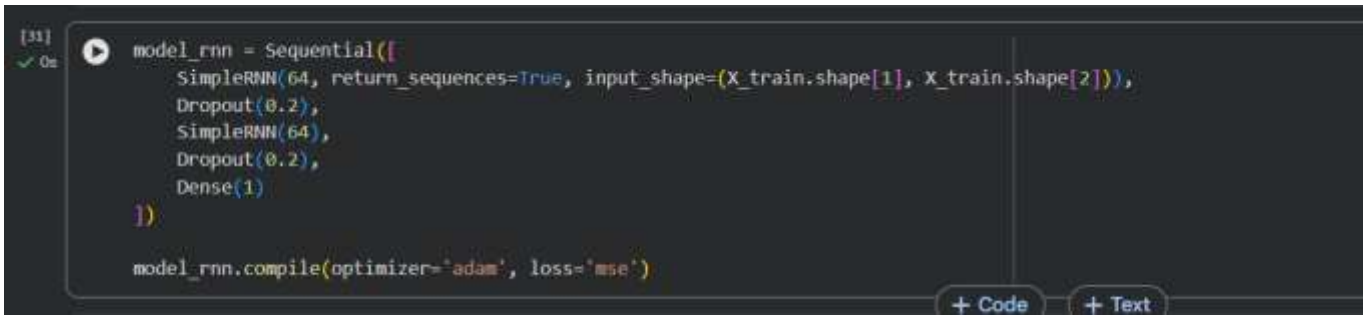
Dropout(0.2),

SimpleRNN(64),

```
    Dropout(0.2),
    Dense(1)
])

model_rnn.compile(optimizer='adam', loss='mse')
```

#screenshot

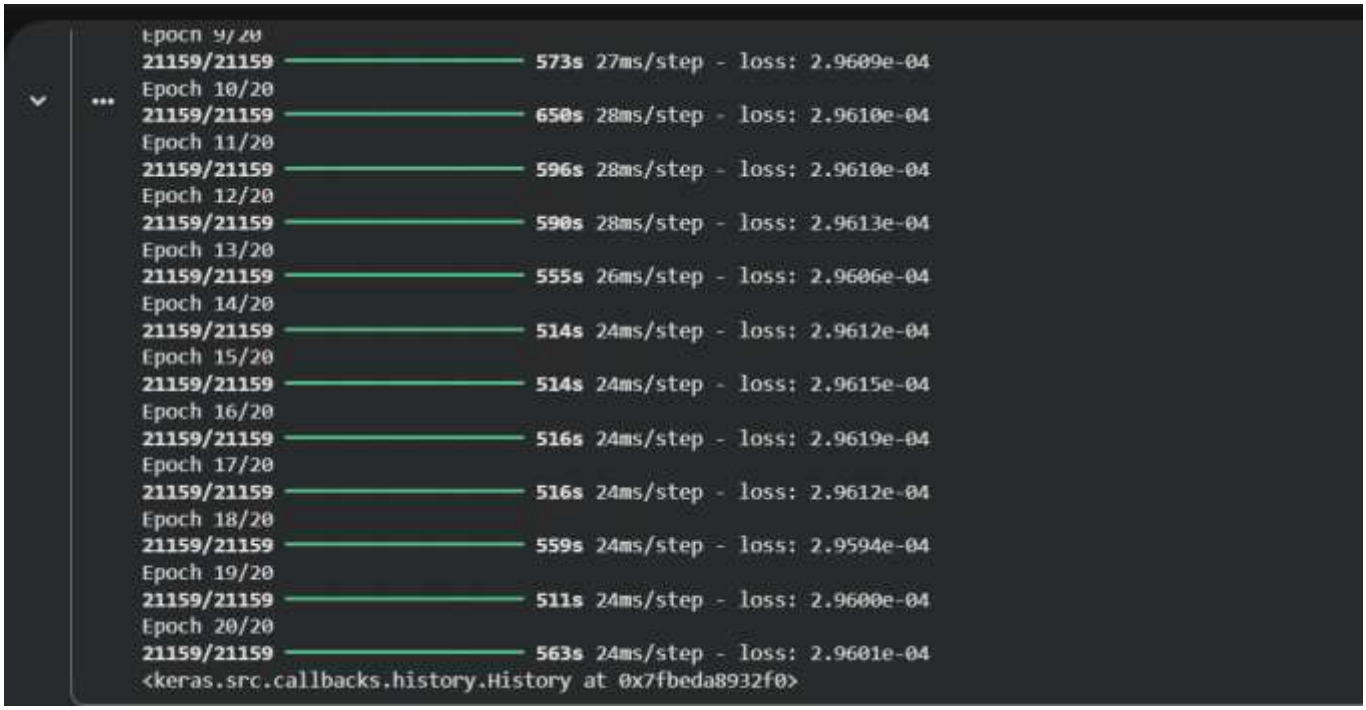
#Train



#code

```
model_rnn.fit(X_train, y_train_close, epochs=20, batch_size=32)
```

#Screenshot

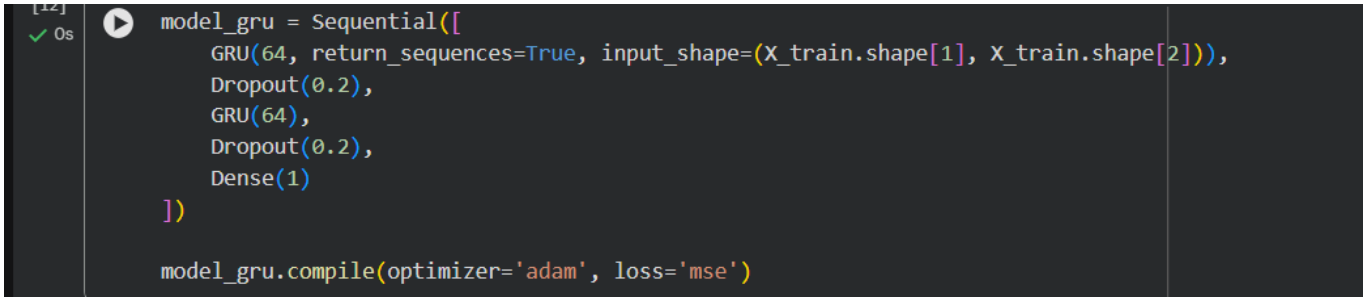


GRU Model (Reference Model)

#code

```
model_gru = Sequential([
   GRU(64, return_sequences=True, input_shape=(X_train.shape[1], X_train.shape[2])),
   Dropout(0.2),
   GRU(64),
   Dropout(0.2),
   Dense(1)
])


model_gru.compile(optimizer='adam', loss='mse')
```

screenshot

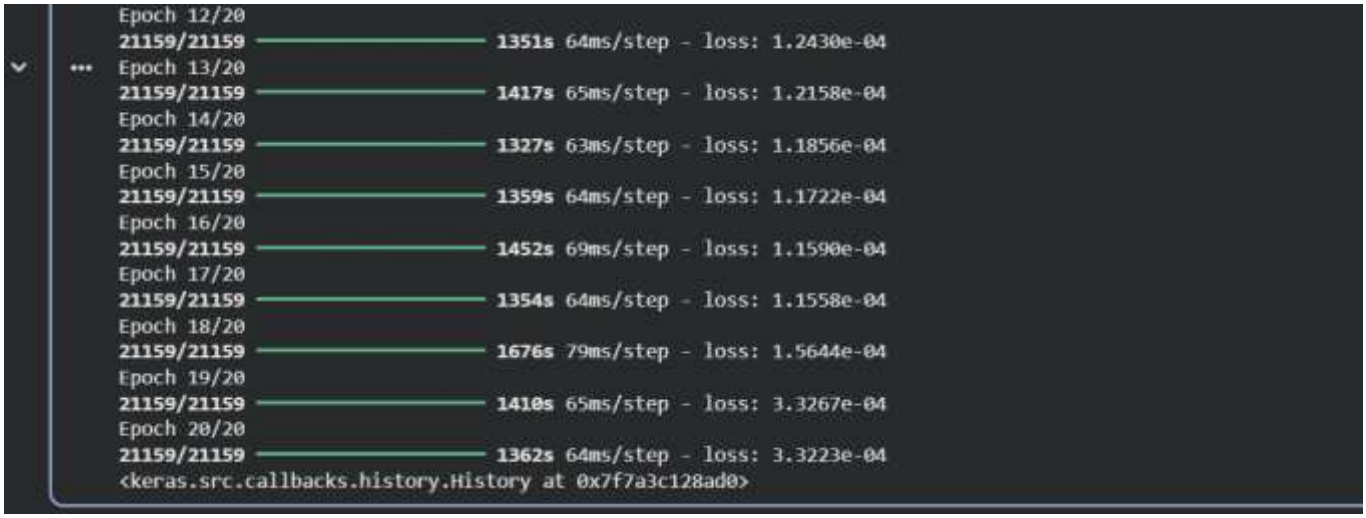


Train

code

```
model_gru.fit(X_train, y_train_close, epochs=20, batch_size=32)
```

screenshot



#Predictions

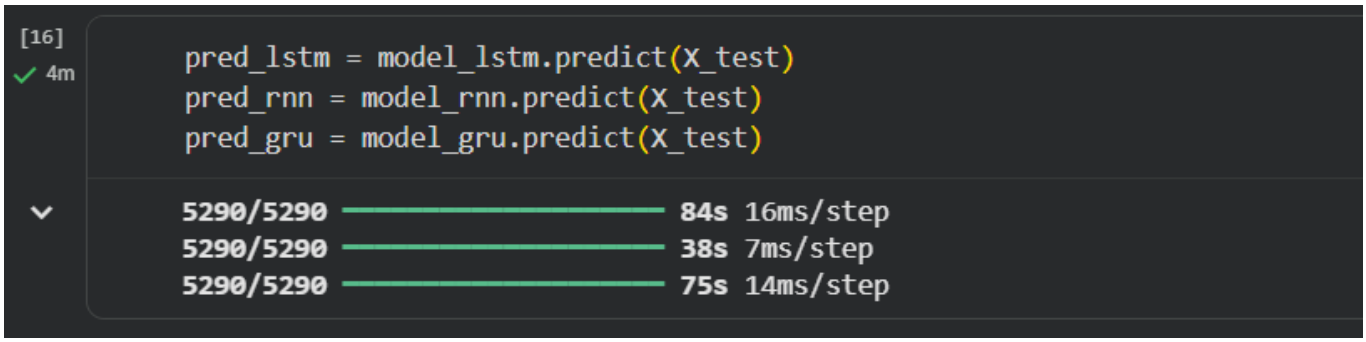
#code

pred_lstm = model_lstm.predict(X_test)

pred_rnn = model_rnn.predict(X_test)

pred_gru = model_gru.predict(X_test)

screenshot



#Evaluation

#code

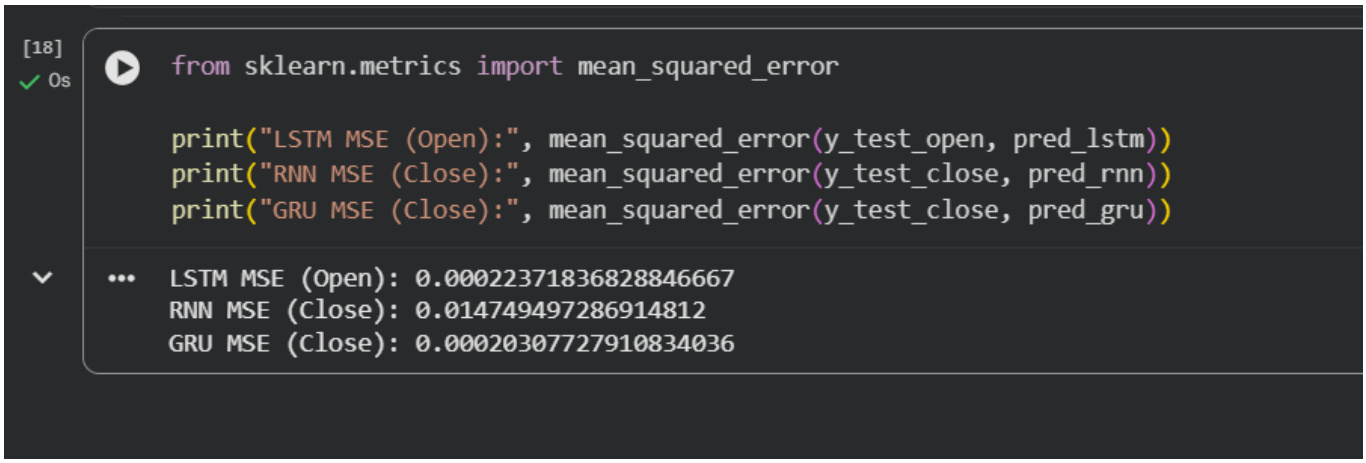
from sklearn.metrics import mean_squared_error

print("LSTM MSE (Open):", mean_squared_error(y_test_open, pred_lstm))

print("RNN MSE (Close):", mean_squared_error(y_test_close, pred_rnn))

print("GRU MSE (Close):", mean_squared_error(y_test_close, pred_gru))

#screenshot



#Visualization

#code

plt.figure(figsize=(12,6))

plt.plot(y_test_open, label='Actual Open')

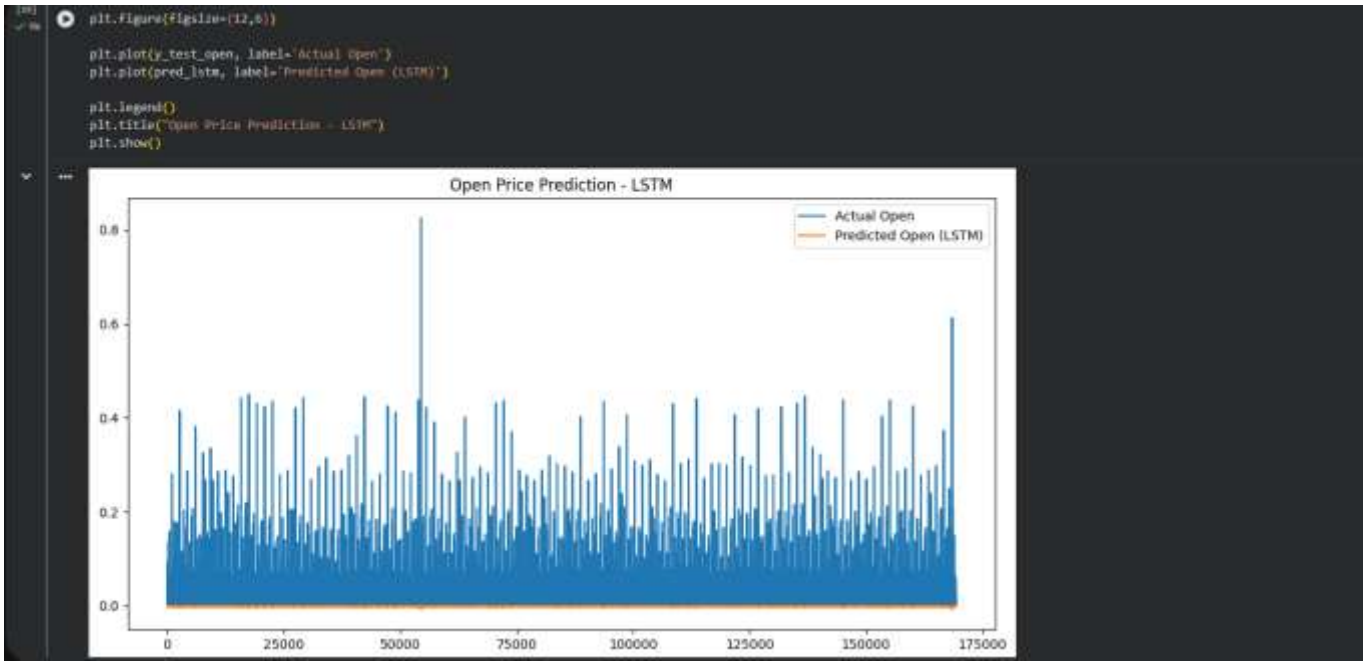
plt.plot(pred_lstm, label='Predicted Open (LSTM)')

plt.legend()

plt.title("Open Price Prediction - LSTM")

plt.show()

#screenshot



conclusion

- LSTM gives best accuracy

- RNN performs worst

- GRU is faster and efficient